

QCrypt – Detailed Project Explanation

Name: M.AbuBakar | **Roll No:** BSCS25109 | **Course:** Object Oriented Programming (C++)

1. What We Did in This Project

We built **QCrypt**, a complete Classical & Quantum Cryptography Engine with a WhatsApp-style graphical messenger interface. The project demonstrates every major OOP concept taught in a second-semester CS course — inheritance, polymorphism, encapsulation, abstraction, templates, operator overloading, dynamic memory management, and composition.

What the project actually does:

Encrypts and decrypts messages using six different cipher algorithms:

- **Caesar Cipher** – shifts each letter by a fixed number (key = 7).
- **ROT13 Cipher** – a special Caesar cipher with a fixed shift of 13.
- **Vigenere Cipher** – shifts each letter by a different amount based on a repeating keyword ("KEY").
- **XOR Cipher** – flips bits of each character using a secret byte key (0xAB).
- **RSA Cipher** – uses real public-key cryptography with prime numbers and modular exponentiation.
- **BB84 Quantum Cipher** – simulates a quantum key distribution protocol inspired by real quantum physics.

Provides a graphical WhatsApp-like messenger built with the Raylib library:

- A left sidebar shows three contacts: **Hafiz Abdullah**, **Omer Abdullah**, and **Tayyab Ahmed**.
- Clicking a contact opens their chat window on the right side.
- A dropdown lets you pick which cipher to use before sending a message.
- Each sent message appears as a green chat bubble showing: the cipher name, the encrypted text, and the original (decrypted) text.
- All chat history is automatically saved to text files (`chat_hafiz.txt`, `chat_omer.txt`, `chat_tayyab.txt`) so messages persist between sessions.

Uses zero standard library containers. Instead of `std::string` and `std::vector`, we built our own:

- **MyString** — a complete dynamic string class with its own `char*` buffer, supporting concatenation, comparison, indexing, copy construction, and `operator<<`.
- **MyVector<T>** — a template-based dynamic array supporting `push_back()`, `operator[]`, `size()`, copy construction, and automatic resizing.

Demonstrates every form of inheritance: single, multi-level, hierarchical, and abstract base classes — totaling **37 classes** with a maximum inheritance depth of 3.

2. How We Built This Project

Phase 1 — Crypto Engine (Backend)

We started by designing the **class hierarchy** on paper (using UML diagrams in `.drawio` files). The core idea was:

*"Every encryption algorithm is a **Cipher**. Every Cipher must implement `encrypt()`, `decrypt()`, `showInfo()`, and `getName()`. We don't care how each one works internally — we just call the same interface."*

This is the foundation of **polymorphism**.

Step-by-step construction:

Built `MyString` and `MyVector<T>` first, since every other class depends on dynamic strings and arrays. These were built using raw `new` and `delete` operators — no standard library.

Created the abstract base class `Cipher` with four pure virtual functions. This means no one can create a `Cipher` object directly — you *must* create a specific cipher like `CaesarCipher`.

Created intermediate abstract classes (`SubstitutionCipher`, `PolyalphabeticCipher`, `BitwiseCipher`, `AsymmetricCipher`, `QuantumCipher`) to group related ciphers and share common logic.

Implemented each concrete cipher (`CaesarCipher`, `ROT13Cipher`, `VigenereCipher`, `XORCipher`, `RSACipher`, `BB84Cipher`) — each one overrides the pure virtual functions with its own algorithm.

Built helper modules: `MathEngine` / `ModularArithmetic` for RSA math, `PrimeEngine` for prime number generation, `KeyGenerator` / `PrimeKeyGenerator` / `SimpleKeyGenerator` for key creation, and the full quantum simulation layer (`Alice`, `Bob`, `Eve`, `QuantumChannel`, `Basis`).

Created `CryptoManager` — the polymorphic container. It stores an array of `Cipher*` pointers and can call `encrypt()` / `decrypt()` on any of them without knowing their actual type.

Phase 2 — GUI Messenger (Frontend)

Created `Contact` and `Message` data classes to represent a friend and a chat message.

Built `MessengerGUI` using the Raylib graphics library. This class owns the render loop (runs at 60 FPS), draws the sidebar, header, chat bubbles, input bar, and cipher dropdown every frame.

Implemented file I/O so that each contact's chat history is saved to a `.txt` file using a pipe-delimited format: `CipherName|EncryptedText|PlainText`.

Loaded the Segoe UI font from Windows to replace Raylib's default pixelated font, making the text crisp and readable.

Build System

The project uses a `Makefile` that compiles all `.cpp` files and links against Raylib (`-lraylib -lgdi32 -lwinmm -lopengl32`). The entry point is `main.cpp`, which creates a `MessengerGUI` object, initializes the Raylib window, and enters the game loop.

3. Which Classes Are Used, Why, and How They Work

3.1 Custom Data Structures

Class	Purpose	How It Works
MyString	Replaces <code>std::string</code> . Stores text as a dynamic <code>char*</code> buffer.	Internally maintains a <code>char*</code> data pointer and <code>int len</code> . When you do <code>str1 + str2</code> , it allocates a new buffer of size <code>len1 + len2 + 1</code> , copies both strings into it, and returns a new <code>MyString</code> . The destructor calls <code>delete[]</code> data to prevent memory leaks. Supports <code>operator=</code> , <code>operator+</code> , <code>operator[]</code> , <code>operator==</code> , <code>toUpper()</code> , <code>toLower()</code> , and <code>c_str()</code> (returns raw pointer for C functions).
MyVector<T>	Replaces <code>std::vector</code> . A template-based resizable array.	Maintains a <code>T* arr</code> , <code>int sz</code> (current size), and <code>int cap</code> (capacity). When <code>push_back()</code> is called and <code>sz == cap</code> , it doubles the capacity by allocating a new array, copying all elements, and deleting the old one. This is the classic "amortized O(1)" dynamic array pattern. Used to store <code>Cipher*</code> pointers in <code>CryptoManager</code> , <code>Message</code> objects in chat arrays, and <code>int</code> bits/bases in the BB84 protocol.

3.2 The Cipher Hierarchy

Class	Type	Purpose
Cipher	Abstract base	Defines the interface: <code>encrypt()</code> , <code>decrypt()</code> , <code>showInfo()</code> , <code>getName()</code> — all pure virtual. Every cipher in the project inherits from this.
SubstitutionCipher	Abstract intermediate	Groups ciphers that work by substituting one character for another. Adds <code>shiftChar()</code> helper.
CaesarCipher	Concrete	Shifts each letter by a fixed key (e.g., <code>key=7</code> means <code>A→H</code> , <code>B→I</code>). Formula: $C = (M + key) \bmod 26$.
ROT13Cipher	Concrete	A Caesar cipher with <code>key=13</code> . Special property: encrypting twice returns the original text (since $13+13=26$).
PolyalphabeticCipher	Abstract intermediate	Groups ciphers that use a repeating keyword to vary the shift per character. Adds <code>getKeyChar()</code> .
VigenereCipher	Concrete	Each letter is shifted by the corresponding letter of the keyword "KEY". Builds a 26×26 tabula recta lookup table.
BitwiseCipher	Abstract intermediate	Groups ciphers that operate at the bit level. Adds <code>toHex()</code> / <code>fromHex()</code> converters.
XORCipher	Concrete	XORs each byte of the message with a secret byte key (0xAB). Output is displayed as hexadecimal. XOR is its own inverse: <code>encrypt(encrypt(x)) = x</code> .

Class	Type	Purpose
AsymmetricCipher	Abstract intermediate	Groups ciphers that use a public/private key pair. Adds key storage members.
RSACipher	Concrete	Uses real RSA math: picks primes $p=61$, $q=53$, computes $n=3233$, $\phi(n)=3120$, public key $e=17$, private key $d=2753$. Encrypts each character as $C = M^e \bmod n$.
QuantumCipher	Abstract intermediate	Groups ciphers that use quantum key distribution. Adds protocol and qubits members.
BB84Cipher	Concrete	Simulates the BB84 quantum key distribution protocol using Alice, Bob, and a quantum channel. Derives a shared secret key, then uses it for XOR encryption.

3.3 Math & Key Generation

Class	Purpose
MathEngine	Abstract base with <code>compute(a, b, mod)</code> pure virtual.
ModularArithmetic	Implements fast modular exponentiation ($a^b \bmod m$) using the square-and-multiply algorithm. Also provides GCD (Euclidean algorithm) and modular inverse (Extended Euclidean). Used by RSA.
PrimeEngine	Implements trial division primality testing and random prime generation.
KeyGenerator	Abstract base with <code>generate()</code> and <code>show()</code> pure virtual.
PrimeKeyGenerator	Generates the RSA key pair: finds p and q , computes n , $\phi(n)$, e , and d .
SimpleKeyGenerator	Stores a simple shift value or keyword string for Caesar/Vigenere.

3.4 Quantum BB84 Layer

Class	Purpose
Basis	Abstract base for quantum measurement bases.
RectilinearBasis	Measures photons at 0° and 90° — correctly reads bits encoded in the rectilinear basis.
DiagonalBasis	Measures photons at 45° and 135° — correctly reads bits encoded in the diagonal basis.
QuantumChannel	Abstract base for photon transmission. Has pure virtual <code>transmit()</code> .
IdealChannel	Passes photons unchanged — no noise, no eavesdropper.
NoisyChannel	Randomly flips 10% of photons to simulate real-world quantum noise.
EavesdroppedChannel	An eavesdropper (Eve) intercepts and re-transmits, causing ~25% error rate.
Party	Abstract base for protocol participants. Has <code>prepare()</code> and <code>measure()</code> .

Class	Purpose
Alice	The sender — generates random bits and random bases, encodes them as photon polarizations, and sends through a channel.
Bob	The receiver — picks random measurement bases, receives photons, and measures them.
Eve	The eavesdropper — intercepts photons with her own random basis, disturbing the quantum state.
RandomGen	A simple pseudo-random number generator using a linear congruential formula. Produces random bits for Alice, Bob, and Eve.

3.5 Application Layer

Class	Purpose
CryptoManager	The polymorphic engine. Holds a <code>MyVector<Cipher*></code> . When the GUI calls <code>getCipher(i)</code> , it returns a <code>Cipher*</code> — the caller never knows if it's a Caesar, RSA, or BB84. This is runtime polymorphism in action.
Contact	Stores a friend's name, phone number, and chat file path. Encapsulates personal data behind getter methods.
Message	Stores a single chat message: the original plain text, the encrypted text, and the cipher name used.
MessengerApp	The console-based fallback app class. Manages contact list and chat output in the standard console using <code>cout</code> .
MessengerGUI	The Raylib-powered GUI class. Owns the 60 FPS render loop, draws the WhatsApp-style interface, handles mouse clicks and keyboard input, calls the cipher engine to encrypt messages, and manages file I/O for chat persistence.

4. Flow of the Project

Here is the complete execution flow from the moment you run the .exe:

```
main.cpp
|
|■■■ Creates MessengerGUI object
|■   ■■■ Constructor initializes 3 Contact objects (Hafiz, Omer, Tayyab)
|■   ■■■ Creates CryptoManager
|■   ■   ■■■ addCipher(new CaesarCipher(7))
|■   ■   ■■■ addCipher(new ROT13Cipher())
|■   ■   ■■■ addCipher(new VigenereCipher("KEY"))
|■   ■   ■■■ addCipher(new XORCipher(0xAB))
|■   ■   ■■■ addCipher(new RSACipher())           ← generates RSA keys (p=61, q=53, e=17, d=2753)
|■   ■   ■■■ addCipher(new BB84Cipher(128,42)) ← creates Alice & Bob objects
|■   ■■■ Sets WhatsApp color palette
|■   ■■■ Loads saved chat history from .txt files
|■
|■■■ Calls init()
|■   ■■■ Opens Raylib window (1000x700 pixels)
|■   ■■■ Sets 60 FPS target
|■   ■■■ Loads Segoe UI font from C:\Windows\Fonts
|■
```

```

■■■ Calls run() – THE MAIN LOOP (repeats 60 times per second)
■
■   ■■■ handleInput()
■       ■■■ Check: Did user click a contact in the sidebar? → switch active chat
■       ■       ■■■ Check: Did user click the cipher dropdown? → toggle it
■       ■       ■■■ Check: Did user click SEND button? → call sendCurrentMessage()
■       ■       ■■■ Check: Did user press Enter key? → call sendCurrentMessage()
■       ■       ■■■ Check: Did user type a character? → append to inputBuffer
■       ■       ■■■ Check: Did user press Backspace? → remove last character
■       ■       ■■■ Check: Did user scroll mouse wheel? → adjust scrollOffset
■       ■
■       ■■■ BeginDrawing() – start frame
■       ■       ■■■ drawSidebar() – contact list with names, phones, message counts
■       ■       ■■■ drawChatHeader() – shows active contact name and phone
■       ■       ■■■ drawChatMessages() – renders green chat bubbles with encrypted/decrypted text
■       ■       ■■■ drawInputBar() – text field, cipher selector, SEND button
■       ■       ■■■ drawCipherDropdown() – popup list of all 6 ciphers (if open)
■       ■■■ EndDrawing() – display frame on screen
■
■■■ Calls close()
    ■■■ Unloads font
    ■■■ Closes Raylib window

```

Destructor Chain (when program ends):

```

~MessengerGUI()
    ■■■ delete manager (CryptoManager)
        ■■■ for each Cipher* → delete
            ■■■ ~CaesarCipher() → delete keyGen (SimpleKeyGenerator)
            ■■■ ~ROT13Cipher() → delete keyGen
            ■■■ ~VigenereCipher() → delete keyGen
            ■■■ ~XORCipher() → delete keyGen
            ■■■ ~RSACipher() → delete keyGen, delete mathEngine
            ■■■ ~BB84Cipher() → delete alice, delete bob, delete channel

```

5. What Happens When You Send a Message (Step-by-Step Sequence)

Let's trace what happens when you type "Hello World" and click **SEND** with the **Vigenere Cipher** selected:

Step 1 — User Types the Message

Every frame (60 times/second), `handleInput()` checks if any keyboard key was pressed using Raylib's `GetCharPressed()`. Each character (H, e, l, l, o, etc.) is appended one-by-one to `inputBuffer[]` (a char array of size 256). The variable `inputLen` tracks how many characters have been typed.

Step 2 — User Clicks SEND (or presses Enter)

`handleInput()` detects either a mouse click on the SEND button rectangle or the Enter key being pressed. It calls `sendCurrentMessage()`.

Step 3 — Get the Selected Cipher

```
Cipher* cipher = manager->getCipher(selectedCipher);
```

`selectedCipher` is an integer (0–5) representing which cipher the user picked from the dropdown. `getCipher()` returns a `Cipher*` pointer. **The GUI code has no idea** whether this pointer points to a `CaesarCipher`, `RSACipher`, or

BB84Cipher — it only knows it has an `encrypt()` method. This is **runtime polymorphism**.

Step 4 — Wrap the Input in MyString

```
MyString message(inputBuffer);
```

A `MyString` object is constructed from the raw `char[]` buffer. This allocates a new `char*` on the heap, copies the characters, and stores the length.

Step 5 — Encrypt the Message

```
MyString encrypted = cipher->encrypt(message);
```

The virtual function call `cipher->encrypt()` is dispatched at runtime to the correct derived class. For **Vigenere Cipher**, this calls `VigenereCipher::encrypt()`:

Converts the message to uppercase: "HELLO WORLD".

Uses the keyword "KEY", repeating it: K-E-Y-K-E-Y-K-E-Y-K-E.

For each letter, looks up the tabula recta (a 26×26 table):

- H + K → row K (10), column H (7) → R
- E + E → row E (4), column E (4) → I
- L + Y → row Y (24), column L (11) → J
- ... and so on for each letter.

Spaces and non-alphabetic characters pass through unchanged.

Returns the encrypted `MyString`, e.g., "RIJVS AMBPN".

Step 6 — Create a Message Object

```
Message msg(message, encrypted, cipher->getName());
```

A `Message` is created with three `MyString` fields:

- `plainText` = "Hello World" (original)
- `encryptedText` = "RIJVS AMBPN" (encrypted)
- `cipherName` = "Vigenere Cipher" (which cipher was used)

Step 7 — Store in Memory

```
chats[activeContact].push_back(msg);
```

The message is added to `chats[activeContact]`, which is a `MyVector<Message>`. If the internal array is full, `MyVector` automatically doubles its capacity, allocates a new array, copies everything over, and deletes the old one.

Step 8 — Save to File

```
saveMessage(activeContact, msg);
```

Opens the contact's chat file (e.g., `chat_hafiz.txt`) in append mode and writes:

```
Vigenere Cipher|RIJVS AMBPN|Hello World
```

This pipe-delimited format allows `loadChat()` to reconstruct the messages when the app starts next time.

Step 9 — Clear Input & Auto-Scroll

The `inputBuffer` is cleared (`inputBuffer[0] = '\0', inputLen = 0`), and the `scrollOffset` is adjusted so the chat view automatically scrolls down to show the newest message.

Step 10 — Render the Bubble

On the very next frame, `drawChatMessages()` iterates through `chats[activeContact]` and draws a green rounded rectangle for each message. Inside each bubble, it renders:

- Line 1: "Vigenere Cipher" (in teal color)
 - Line 2: "Encrypted: RIJVS AMBPN" (in dark gray)
 - Line 3: "Decrypted: Hello World" (in black)
-

6. RSA and BB84 — How They Encrypt and Decrypt in Detail

6.1 RSA Cipher — Detailed Working

RSA is fundamentally different from Caesar or Vigenere. Instead of shifting letters, it uses **number theory** — prime numbers and modular exponentiation.

Key Generation (happens once when RSACipher is created)

The `PrimeKeyGenerator` class performs these steps:

```
Step 1: Pick two prime numbers
        p = 61, q = 53

Step 2: Compute n = p × q
        n = 61 × 53 = 3233

Step 3: Compute Euler's totient  $\phi(n) = (p-1) \times (q-1)$ 
         $\phi(3233) = 60 \times 52 = 3120$ 

Step 4: Choose public exponent e such that  $1 < e < \phi(n)$  and  $\gcd(e, \phi(n)) = 1$ 
        e = 17 (17 and 3120 share no common factors)

Step 5: Compute private exponent d =  $e^{-1} \bmod \phi(n)$ 
        This uses the Extended Euclidean Algorithm:
        d = modInverse(17, 3120) = 2753
        Verify:  $17 \times 2753 = 46801 = 15 \times 3120 + 1$  ✓ (remainder is 1)

Result:
        Public Key = (e=17, n=3233) — used to ENCRYPT
        Private Key = (d=2753, n=3233) — used to DECRYPT
```

Encryption — When You Send "Hello World"

When you type "Hello World" and select RSA, the `RSACipher::encrypt()` function processes **each character individually**:

```
For each character:
1. Get its ASCII value: 'H' = 72, 'E' = 69, 'L' = 76, etc.
2. Compute:  $C = M^e \bmod n = M^{17} \bmod 3233$ 
3. Format as [C]
```

```
Example with 'H':
M = 72 (ASCII of 'H')
C =  $72^{17} \bmod 3233$ 
```

The `modPow()` function computes this efficiently using square-and-multiply:

17 in binary = 10001, so $72^{17} = 72^{16} \times 72^1$
 $C = (2280 \times 72) \bmod 3233 = 164160 \bmod 3233 = 2790$

Full message: "HELLO WORLD" → "[2790][1231][2429][2429][2338][1828][499][2338][1063][2429][939]"

Decryption

$$C = 2790$$
$$M = C^d \bmod n = 2790^{2753} \bmod 3233$$

Result: $M = 72 \rightarrow \text{char}(72) = \text{'H'}$ ✓

Why RSA works: The mathematical relationship $(M^e)^d \bmod n = M$ holds because of Euler's theorem. Without knowing d (which requires knowing p and q), an attacker would need to factor $n = 3233$ into its primes — trivial for small numbers, but computationally infeasible for real RSA with 2048-bit keys.

```

STEP 1: Alice Prepares (Alice::prepare)
Alice generates 128 random bits and 128 random bases using her RandomGen:
  Bits:    [0, 1, 1, 0, 1, 0, 0, 1, 1, 0, ...]    (128 values)
  Bases:   [0, 1, 0, 1, 0, 0, 1, 1, 0, 1, ...]    (128 values)

```

Base 0 = Rectilinear (+): encodes bit 0 as 0° and bit 1 as 90°
Base 1 = Diagonal (×): encodes bit 0 as 45° and bit 1 as 135°

STEP 2: Bob Prepares (Bob::prepare)

Bob independently generates 128 random measurement bases:

Bases: [1, 0, 0, 1, 1, 0, 0, 1, 0, 0, ...] (128 values)

Bob does NOT know Alice's bases – this is the key to the protocol.

STEP 3: Alice Sends Photons Through Channel

For each of the 128 qubits:

Alice calls: channel->transmit(photon)

With IdealChannel: the photon arrives unchanged.

With NoisyChannel: 10% chance the photon gets randomly flipped.

With EavesdroppedChannel: Eve intercepts and re-measures, disturbing ~25%.

STEP 4: Bob Measures Each Photon

Bob receives each photon and measures it with his chosen basis:

If Bob's basis MATCHES Alice's basis → Bob gets the CORRECT bit (100%)

If Bob's basis DIFFERS from Alice's → Bob gets a RANDOM bit (50/50)

Example:

Qubit #0: Alice bit=0, Alice base=0(+), Bob base=1(×) → MISMATCH → random

Qubit #1: Alice bit=1, Alice base=1(×), Bob base=0(+) → MISMATCH → random

Qubit #2: Alice bit=1, Alice base=0(+), Bob base=0(+) → MATCH → Bob gets 1 ✓

Qubit #3: Alice bit=0, Alice base=1(×), Bob base=1(×) → MATCH → Bob gets 0 ✓

STEP 5: Sift the Key (Basis Reconciliation)

Alice and Bob publicly compare their BASES (not the bits!):

"I used base 0, you used base 1 → discard"

"I used base 0, you used base 0 → keep!"

The code does:

```
for each qubit i:  
    if (aliceBases[i] == bobBases[i])  
        siftedKey.push_back(aliceBits[i]);
```

Statistically, about 50% of the 128 qubits will match → ~64 shared key bits.

STEP 6: Error Rate Check (Eavesdrop Detection)

Alice and Bob sacrifice 25% of the sifted key bits to check for errors:

They compare ~16 bits from the sifted key.

If the error rate > 20%, it means someone (Eve) was eavesdropping.

→ The protocol aborts and retries with new random values.

If the error rate ≤ 20%, the remaining key bits are trusted.

STEP 7: XOR Encryption Using the Shared Key

Once a good key is established (e.g., 48 bits: [1,0,1,1,0,0,1,0,...]):

```

For each character at position i:
    key_bit = sharedKey[i % keyLength]
    encrypted_char = original_char XOR key_bit

```

... and so on for each character.

DECRYPTION

```
73 XOR 1 = 72 → 'H' ✓
69 XOR 0 = 69 → 'E' ✓
```

BB84 demonstrates **multi-level inheritance** (`BB84Cipher` $\rightarrow$ `QuantumCipher` $\rightarrow$ `Cipher`), **composition** (`BB84Cipher` *has-a* `Alice`, `Bob`, and `QuantumChannel`), and **dynamic dispatch** (the channel's `transmit()` method resolves at runtime to either `IdealChannel`, `NoisyChannel`, or `EavesdroppedChannel`).

#	Class	Type	Parent	Key OOP Concept
1	MyString	Utility	—	Operator overloading, encapsulation, dynamic memory
2	MyVector<T>	Utility	—	Templates (generics), dynamic memory
3	Cipher	Abstract	—	Pure virtual functions, polymorphism
4	SubstitutionCipher	Abstract	Cipher	Single inheritance
5	CaesarCipher	Concrete	SubstitutionCipher	Composition with SimpleKeyGenerator
6	ROT13Cipher	Concrete	SubstitutionCipher	Hierarchical inheritance
7	PolyalphabeticCipher	Abstract	Cipher	Single inheritance
8	VigenereCipher	Concrete	PolyalphabeticCipher	Tabula recta, keyword shifting

#	Class	Type	Parent	Key OOP Concept
9	BitwiseCipher	Abstract	Cipher	Hex encoding utilities
10	XORCipher	Concrete	BitwiseCipher	Bitwise XOR, self-inverse
11	AsymmetricCipher	Abstract	Cipher	Public/private key concept
12	RSACipher	Concrete	AsymmetricCipher	Modular exponentiation, composition
13	QuantumCipher	Abstract	Cipher	Quantum protocol base
14	BB84Cipher	Concrete	QuantumCipher	Multi-level inheritance (depth 3)
15	MathEngine	Abstract	—	Pure virtual compute()
16	ModularArithmetic	Concrete	MathEngine	Fast exponentiation, GCD
17	PrimeEngine	Concrete	MathEngine	Primality testing
18	KeyGenerator	Abstract	—	Pure virtual generate()
19	PrimeKeyGenerator	Concrete	KeyGenerator	RSA key pair generation
20	SimpleKeyGenerator	Concrete	KeyGenerator	Shift/keyword storage
21	Basis	Abstract	—	Quantum measurement
22	RectilinearBasis	Concrete	Basis	0°/90° measurement
23	DiagonalBasis	Concrete	Basis	45°/135° measurement
24	QuantumChannel	Abstract	—	Pure virtual transmit()
25	IdealChannel	Concrete	QuantumChannel	No disturbance
26	NoisyChannel	Concrete	QuantumChannel	10% noise
27	EavesdroppedChannel	Concrete	QuantumChannel	Eve intercepts
28	Party	Abstract	—	Protocol participant
29	Alice	Concrete	Party	Sender — prepares photons
30	Bob	Concrete	Party	Receiver — measures photons
31	Eve	Concrete	Party	Eavesdropper
32	RandomGen	Utility	—	Pseudo-random number generator
33	CryptoManager	Manager	—	Polymorphic cipher container
34	Contact	Data	—	Encapsulated contact info
35	Message	Data	—	Encapsulated message data

#	Class	Type	Parent	Key OOP Concept
36	MessengerApp	App	—	Console-based messenger loop
37	MessengerGUI	GUI	—	Raylib render loop, event handling